

Axelspace

Coding Assignment for Embedded Systems Software Engineer

Notice: Please do not share this assignment with anyone except Axelspace employees.

Part 1

Imagine that as a part of the team you were assigned to develop and test two functions:

1. For packet detection
2. For packet parsing

Functions have to be compatible with the provided interface to make integration possible.

Code Submission

Code:

- Please write the code as if it was a real work assignment.
- Please use C++11 or higher, and C++ standard libraries
- Please focus on the **portability** aspect. If possible, code should be **portable between CPU architectures, compilers and OSes**.
- We will give additional points for low footprint and determinism of the solution.

Tests:

- Provide unit tests proving correctness of your solution
- Tests may be written for any OS, compiler and in any language as well as using framework of your choice. Please send a short HOWTO explaining how to build and run tests.

Communication protocol description

Basic conditions:

- Packet size is fixed to 12 bytes
- Every packet starts with a “magic” header
- Byte order of data in the packet is from most significant to least significant
- You cannot assume that all incoming bytes are correct, consistent packets.

There may be other types of data between packets, for example:

[packet] [packet] [control sequence] [packet] [spurious bytes] [control sequence] [packet]

Packet layout:

Magic header [4B] 0xABBACFFC	Secondary Header [2B]	Reserved [1B] 0xFF	Payload [4B]	Checksum [1B]
------------------------------------	--------------------------	--------------------------	-----------------	------------------

Secondary header layout:

Protocol version [3 bit]	Subsystem ID [5 bit]	Component ID [5 bit]	Telemetry type [3 bit]
-----------------------------	-------------------------	-------------------------	---------------------------

Magic header:

- 0xABBACFFC marks the start of a packet

Secondary header:

- Protocol versions:
 - Supported protocol versions: 1, 2
 - Both versions have the same packet layout.
- Valid subsystem IDs:
 - 1 - AOCS
 - 3 - CDH
 - 5 - COM
- Valid component IDs for each subsystem:
 - AOCS: 20, 21, 22, 23, 30, 31
 - CDH: 0
 - COM: 1, 2, 10, 20
- Valid telemetry type:
 - 1 - temperature (implies data of type IEEE 754 single precision floating-point)

Reserved field:

- One byte reserved for future protocol expansion.
 - Must be set to 0xFF

Payload:

- If telemetry types is ‘1’ (temperature), the value in the 4 bytes of payload is IEEE 754 single precision floating-point.

Checksum:

- Simple exclusive-or (xor) of all bytes from the start of secondary header to the end of payload (including reserved field).

Interface description

Based on the example code fragments below, please design and implement **Parser** class.

tlm_parser.h:

```
class Parser {  
    // Implement me  
};
```

tlm.h:

```
typedef struct {  
    uint8_t subsys_id;  
    uint8_t compo_id;  
    float temperature;  
} CompoTlm_t;
```

tlm_proc.cpp:

```
...  
...  
while ( 1 ) {  
    while (iface.existRcvData()) {  
        uint8_t byte = iface.getBytes();  
        // Parse incoming bytes and detect packet  
        if (parser.detectPkt(byte)) {  
            // One, correct packet has been detected  
            CompoTlm_t tlm = {};  
            int err = parser.extractData(tlm);  
            if (err == 0 ) {  
                // Pass telemetry info to the upper layer  
                system.tlm.update(tlm);  
            } else {  
                // Error handling, etc.  
                ...  
                ...  
            }  
        }  
    }  
    system.time.sleep_ms( 100 );  
}  
...  
...
```

- **iface** is an object of a class that is used to communicate with the peripheral devices
 - **existRcvData()** method returns `true` if there are bytes in the Rx FIFO
 - **getBytes()** method fetches one byte from the Rx FIFO
- **parser** is an object of a class that you are asked to implement
- **system** is an object of a class that communicates with the hypothetical OS
 - **tlm** is an object used to process system's telemetry.
 - **update()** method can be used to send latest telemetry to the system control

Part 2

1. What is your favorite **IPC** mechanism and why?
2. Imagine a program that realizes a very simple task a very large number of times; for example, count up to 1 billion, one by one:

```
volatile int counter = 0;

int main(int argc, char** argv)
{
    size_t nb_iter = 1000000000;

    for (int i=0; i<nb_iter; i++)
    {
        counter ++;
    }

    printf("counter: %ld / %ld\n", counter, nb_iter);
    return 0;
}
```

On a recent multi-core CPU, how could this program be enhanced to run **faster**? Please use C++11 or higher and C++ standard libraries.